Northstar Supply Owner Guide

What this guide covers. This is a plain-language guide to running, demonstrating and safely updating the two-page web application. Page one reads product data from XML. Page two asks WeatherAPI for live weather data in JSON.

What has been delivered

File	Purpose
index.html	The XML product catalogue page.
Products.xml	The editable product data file with ten products.
products.js	Reads the XML file, displays products and handles product-code searches.
weather.html	The weather-search page.
weather.js	Calls WeatherAPI and displays the JSON response.
style.css	Shared colours, layout and responsive design for both pages.

How to run and demonstrate the application

Use a local web server rather than opening the HTML file directly. This is important because browsers can block a page from reading Products.xml when it is opened from the file system.

Open the project folder in Visual Studio Code.

Install the Live Server extension if it is not already installed.

Right-click index.html and choose Open with Live Server.

On the Products page, show the ten product cards. Search for 101-01 to show a successful search, then search for 999-99 to show the helpful not-found message.

Open the Weather page. Enter a city and a WeatherAPI key, then select Get weather.

How the XML product page works

Products.xml is a small structured data file. Each product has a unique code attribute and five data fields: category, name, description, quantity and unit price. products.js uses fetch to load that file, DOMParser to read the XML, and JavaScript to turn each product into a visible card.

To add a product, copy one whole product line in Products.xml, paste it before the closing products tag, and change every value. The code must be unique and use exactly three digits, a hyphen, then two

digits.

```
<product code="111-30"><category>Pantry</category><name>Example
Item</name><description>Short
description.</description><quantity>10</quantity><unitPrice>2.50</unitPrice></product>
```

Important: quantity must be zero or more. Unit price must be zero or more. Keep the opening and closing tags exactly as they are.

How the weather page works

The weather page makes a GET request to WeatherAPI using the city name and API key entered in the form. WeatherAPI replies in JSON, which weather.js checks and then displays as temperature, feels-like temperature, humidity and wind speed.

The application deliberately does not store an API key in the source code. This prevents a key being accidentally shared with the project. For the assignment demo, paste the key into the form when needed. A production website would send this request through a secure server instead.

Useful code comments

The JavaScript contains short comments above the most important ideas: converting XML into JavaScript objects, rendering product cards, safely building the weather URL, and protecting the API key. Read the comment immediately before or above the relevant function, then read the next few lines to see it in action.

Quick troubleshooting

No products appear: Make sure the page is running through Live Server and Products.xml is still in the project folder.

Product search says invalid format: Use a code such as 101-01, with three digits, a hyphen and two digits.

Weather request fails: Check that the city is spelled correctly, the internet is available and the WeatherAPI key is active.

Weather request says key is invalid: Create or copy a valid key from WeatherAPI and paste it again. Do not add it permanently to the JavaScript file.

Assignment checklist for Part A

• Two linked web pages are present.

• The XML page has ten products, displays all product details and searches by code.

• The weather page validates both fields, makes the required GET request and handles errors.

• The pages are responsive, readable and include helpful feedback messages.

• Keep screenshots or a short demonstration recording as evidence of the working features.